Homomorphic Encryption for Practical Secure Computation: Case Studies in Drone Navigation and Decision Tree Inference

Andrew Quijano
Dept. of Computer Science and Engineering
New York University
New York, NY, USA
andrew.quijano@nyu.edu

Abstract*—Although data, such as precise location information or protected health information (PHI), hold immense value, extracting useful output often requires processing by external parties, inherently raising significant privacy concerns. However, conventional encryption paradigms fundamentally restrict data processing to the plaintext domain, compelling users to place complete trust in third-party entities for secure key management and the handling of unencrypted sensitive information, thus risking data compromise. There are other approaches such as garbled circuits that can allow for some secure data processing, but it requires either extensive preprocessing or the specific operations on encrypted data is limited. Homomorphic encryption (HE) promises to give users strong privacy guarantees while allowing them to outsource computation over their data to external parties. In this work, we apply HE to two real-world applications: collision avoidance for drone systems and outsourced inference for tall sparse decision trees. In the first contribution to drone collision avoidance, we implemented the line intersection algorithm entirely using HE. The simplicity of the algorithm improved the performance compared to other state-of-the-art approaches. Through careful design of the HE version of the line intersection algorithm, we can achieve about 30% faster computation time and a 90% reduction in the number of network bytes needed compared to other approaches. As for the second contribution, Joye and Salehi implemented a secure decision tree evaluation protocol utilizing HE that is also resistant to timing attacks. However, in the case of tall and spare decision trees, we show that Joye and Salehi's approach will cause a massive performance decrease. In our work, we split the decision tree model, so that we can minimize the number of HE operations on the evaluation protocol, and it also preserves resistance to timing attacks. Our outsourced sparse decision tree evaluation protocol has on average a performance gain between 30% and 50%. Both our contributions show that HE can be used in practice in important applications to enforce strong privacy guarantees without imposing a massive performance trade-off.

I. Introduction

A. Motivation and Problem Statement

The enduring importance of cryptography in securing sensitive information is undeniable, with its history traced back to ancient ciphers such as the Caesar Cipher and evolving into modern robust algorithms such as AES and RSA. In the contemporary digital age, where the Internet underpins critical services such as financial transactions and healthcare data exchange, encrypting communications is paramount to protecting user privacy and ensuring data privacy.

However, a fundamental challenge persists with traditional encryption schemes: For any computation or analysis to occur on encrypted data, it must first be decrypted. This necessity introduces significant security vulnerabilities. Decryption creates moments of exposure in which sensitive data, now in plaintext, become susceptible to various attacks, such as malware designed to intercept data post-decryption [32]. Furthermore, while algorithms such as RSA and AES are cryptographically strong, their security is intrinsically tied to robust key management. The risk of compromised keys, whether through accidental exposure on public platforms like GitHub or successful breaches leading to private key theft, remains a persistent and significant concern. These vulnerabilities underscore a critical need for new cryptographic paradigms that allow data utility without compromising confidentiality at the point of processing.

To address these limitations, the concept of *homomorphic encryption (HE)* emerged as an interesting solution. HE uniquely allows computations, such as addition or multiplication, to be performed directly on ciphertext, without requiring prior decryption. This capability fundamentally alters the security landscape for outsourced data processing, as it eliminates the reliance on third parties handling sensitive information in its vulnerable plaintext form. By enabling operations on encrypted data, HE inherently mitigates the risks associated with decryption points and key exposure, marking a significant advancement in privacy-preserving computation.

B. Contributions

The preliminary contribution is to extend the work of Quijano and Akkaya [37], who have implemented some homomorphic encryption algorithms such as Veugen's encrypted integer comparison [45], [46] and encrypted division [44]. We separate the homomorphic encryption code used in their work [37] as a library, extend this library, and implement new functionality, such as encrypted multiplication [47], Joye and Salehi's encrypted integer comparison [20], and Veugen's encrypted equality check [31]. Furthermore, we design this

library to be flexible for use in various different projects, further emphasizing the flexibility of homomorphic encryption as a solution for privacy-preserving computation in various fields. In particular, we focus on the prevention of drone collision avoidance [28] and the evaluation of a decision tree [38].

With regard to drones, they are rapidly proliferating in various domains, such as aerial surveillance [8], package delivery [5], and search and rescue operations [7], [29], and many industries are seeing significant advancements. However, data privacy concerns arise as drones are responsible for more services, with one particular instance being the usage of delivery drones. Delivery drones are cost-effective because companies can reduce the costs associated with delivery drivers, fuel, and vehicle maintenance. Drones can also bypass ground traffic and take more direct routes, significantly reducing delivery times compared to traditional delivery methods. Finally, drones can operate outside typical delivery hours, offering faster and more flexible delivery options for consumers. However, as more delivery drones are being used, collision avoidance becomes a serious concern. Traditional collision avoidance mechanisms often involve sharing detailed flight paths between drones, raising significant privacy concerns. This information may include specific home addresses, shopping patterns, and other data related to individual privacy. If the data is compromised, criminals might target specific homes or businesses for theft or other malicious activities, as they know when valuable packages are expected to arrive. Also, it is important to keep this information private between drones owned by competitors, as leaked delivery data could be used to gain insight into a company's supply chain, customer base, and operational practices, leading to potential business espionage.

Given this background, we answer the first research question:

Can we use homomorphic encryption to implement a fast drone collision avoidance mechanism?

We address this question by implementing the line intersection algorithm [15] using homomorphic encryption. Our protocol assumes that two drones that have different owners are about to collide. To avoid over-complicating paths, we assume all paths are in a 2-dimensional plane. We utilize an intersection algorithm that operates on encrypted paths using homomorphic encryption, enabling drones to detect potential collisions without revealing the full path of the other drone. The drones would start this protocol once they are within a range that is reasonable enough to cause a collision. If a collision will occur, the drone will temporarily change its altitude (that is, by modifying the Z coordinate) and return to its default altitude once its path no longer overlaps with the other drone. We observed a **30% faster computation and 90% less bytes sent between both parties compared to the state of the art**.

In regard to our second contribution, machine learning is widely used in various real-world applications, such as managing student data [50], analyzing health data [25][30], and fraud detection in energy consumption [9]. One type of classifier commonly used in machine learning is the Decision Tree (DT). A DT is a classifier that consists of decision nodes and leaves corresponding to an output class. DT evaluations have the advantage of being a fast and computationally inexpensive solution compared to other machine learning techniques. In addition, a DT model is easy to audit as decisions are made through tree traversal. However, as machine learning techniques begin to analyze sensitive data, such as medical records, privacy concerns arise. Privacy is becoming an important issue as there are regulations such as the Health Insurance Portability and Accountability Act (HIPAA) in the United States [18] and the General Data Protection Regulation (GDPR) in the EU [34] that protect sensitive user data. We propose using a *privacy-preserving decision tree (PPDT)* to achieve both accuracy and privacy preservation. A PPDT ensures that the server(s) storing the DT would not have information about the model leaked to the client, and the server is unaware of the client's input or output of the DT model.

Previous work by Joye and Salehi [20] designed a theoretical PPDT classification protocol that used an encrypted integer comparison protocol based on Veugen's [45][46] protocol that has additional protection against timing attacks. To this end, they converted a DT model into a complete binary tree, where all classes are in the bottom level of the tree. This ensures that every evaluation will have the same number of encrypted integer comparisons, so using a timing attack to extract information about the DT is still infeasible. However, the PPDT model proposed in [20] has some issues.

Given this background, we answer the second research question:

Can we improve Joye and Salehi's PPDT such that it no longer requires a complete binary tree but also maintains/improves its security properties?

We extend the theoretical PPDT classification protocol in Joye and Salehi [20] by splitting the PPDT model between multiple entities, which we call *level-sites*, inspired by the fact that the protocol progresses level by level, ending as soon as the client reaches a leaf. Assuming that the user's classification will not always result in traversing the whole tree, we observed a faster evaluation time in our experiments, as we have fewer encrypted comparisons to compute. Each level-site takes less than 1 second to complete the encrypted integer comparison and passing the index to the next level-site. We also show that our approach is resistant to side channel attacks and single-point failures through our decentralized level-site approach. We observed **between 30% and 50% faster computation on average compared to other PPDTs** [52][27].

II. Background

In this section, we briefly summarize the background about homomorphic encryption and the library we use.

A. Homomorphic Encryption

An additive homomorphic encryption scheme such as Paillier [33] or DGK [10][11][12] allows a user to combine two ciphertexts to receive the sum of both ciphertexts. Assume

there are two messages α, α' in the plaintext space $\mathbb{M}$ and we define the encryption of α as $[\![\alpha]\!]$. We use the 'boxplus' operator ($\boxplus$) to denote the addition of two ciphertexts such that upon decryption $[\![(\alpha+\alpha')]\!] = [\![\alpha]\!] \boxplus [\![\alpha']\!]$.

In Paillier [33], we define m as the plaintext message, n is the product of primes p and q, r is a random number and g is a parameter of the encryption scheme, as shown in Eq. 1.

$$[\![m]\!] = g^m r^n \mod n^2 \tag{1}$$

If you multiply two Paillier ciphertexts, the output would be an encrypted addition of the plain-text. A Paillier ciphertext that is exponentiated with a plaintext value would return the encrypted product of the ciphertext and plaintext as in Eq. 2.

$$\begin{aligned} [\![\alpha+\alpha']\!] &= (g^\alpha r^n)(g^{\alpha'} r^n) = g^{\alpha+\alpha'} r^{2n} \\ [\![(\alpha)(\alpha')]\!] &= (g^\alpha r^n)^{\alpha'} = g^{(\alpha)(\alpha')} r^{(\alpha')(n)} \end{aligned} \tag{2}$$

However, Paillier does not have an operation in which one can obtain the product of two ciphertexts. However, using a two-party protocol [47] that takes advantage of the distributive property of multiplication, we can obtain the product of two Paillier [33] ciphertexts. In this protocol, we assume that Alice has $[\![x]\!]$ and $[\![y]\!]$, and generates two random values a and b to additively blind each encrypted value, respectively. Alice sends $[\![(x+a)]\!]$ and $[\![(y+b)]\!]$ to Bob to decrypt, multiply, and return the product. Then Alice could use the following operations to obtain $[\![xy]\!]$, as shown in Eq. 3.

$$[\![xy]\!] = [\![(x+a)(y+b)]\!] \boxplus [\![-bx]\!] \boxplus [\![-ay]\!] \boxplus [\![-ab]\!] \tag{3}$$

Finally, another application of homomorphic encryption is to compare it to encrypted numbers. Assume that there are two t-bit integers held by party A and party B, which are represented in binary form, $x = \sum_{i=0}^{t-1} x_i 2^i$ and $y = \sum_{i=0}^{t-1} y_i 2^i$. The objective of parties A and B, respectively, is to obtain the protocol bits δ_A and δ_B, such that $\delta_A \oplus \delta_B = \mathbb{1}\{x \leq y\}$.

The secure integer comparison problem was first stated in [51], where two millionaires would like to determine who is richer without revealing the amount of their wealth. The first solution to the problem was introduced in [23] and various more advanced approaches such as the implementation of the DGK cryptography system and its comparison protocol [10][11][12]. The high communication cost of these comparison protocols was addressed [45][46] and finally [20] improved them to be resistant to timing attacks.

B. Encryption Library

We utilize a Java library implemented by Quijano and Akkaya [37][36], which implements the encrypted equality check [31], multiplication over two homomorphic ciphertexts [47] and uses the Joye and Salehi encrypted integer comparison protocol [20]. The encrypted integer, multiplication, and equality check can be used for pairs of Paillier [33] ciphertexts or DGK ciphertexts [10][11][12]. We use these functions to implement a privacy-preserving version of a line intersection algorithm [15].

III. Related Work

In this section, we review the related work in both PPDTs and privacy-preserving collision avoidance algorithms.

A. Drone Collision Avoidance

Privacy-preserving collision avoidance has received some recent attention in various contexts. For example, Li et al. [22] developed a multi-robot planning protocol that preserves privacy that would ensure that two robots on a floor would not collide. A garbled circuit is a function represented by a Boolean circuit with logic gates. Using a two-party MPC approach, one party is assigned the role of *garbler* which generates the garbled circuit, while the other party is called the *evaluator*. The inputs of each party are then encrypted, and the *evaluator* processes the garbled gates with the garbled (encrypted) inputs to compute the garbled output. The *evaluator* can then map this garbled output back to the actual output values using the mapping provided by the *garbler* [48]. In Li et al.'s implementation, the output is a Boolean decision on whether an intersection is present somewhere in the complete path. It does not provide exact intersection locations, unlike in our work [28].

Sciancalepore et al. [40] introduce Privacy-Preserving Trajectory Matching (PPTM), a protocol that enables UAVs to discover potential spatial and temporal collisions without sharing sensitive location and timestamp data with untrusted entities. PPTM employs a tree-based algorithm called Incremental Capsule Matching and integrates a lightweight privacy-preserving proximity testing solution for private comparisons. This approach improves path planning efficiency, reduces delivery time, and minimizes energy consumption for UAVs. However, it does not always guarantee accurate solutions. It trades accuracy and performance, unlike in our work, where cryptography allows us to maintain privacy without sacrificing accuracy [28].

Finally, Desai et al. [13] propose a secure MPC for trajectory planning among drones, ensuring collision detection without disclosing sensitive information. It uses matrix representation and matrix addition as an approach to compare trajectories, thus reducing computational complexity and improving performance. The matrix representation involves considering each path to take place on a discrete grid of the region. Each drone's path is represented by a series of continuous cells of value one, where any place the drone will not pass through is given a value zero. The matrix must be consistent between the drones, and so must cover the entire area of possible flight, and cannot handle traffic that is not familiar with the local matrix, such as a drone from another area.

Compared to the approach of Li et al. [22], our approach is faster and lighter on network traffic (as will be shown in Section IV-C), as well as allowing for planning by more than two drones by allowing for deterministic path changes in the event of an intersection rather than random movement. Then, Sciancalepore et al. [40] the most lightweight implementation of their protocol does have false positives and may not be trustworthy in applications where safety is critical such as

drone delivery or military surveillance. Finally, compared to Desai et al. [13], our approach eliminates the need for a discrete matrix of paths possibilities, instead taking GPS coordinates as input. This allows for more detailed planning and also bases computation time solely on the complexity of the path, and not on the size of the delivery area being considered. In addition, a drone coming into an area from a far distance could utilize our method because it does not need to have a common matrix of possible locations.

B. Privacy Preserving Decision Trees (PPDTs)

Several methods have been proposed for the implementation of PPDTs in the literature [42], [6]. Although some of these methods focus on protecting the confidentiality of training data [26], [14], other related works, such as [6] and [41], focus on a secure inference protocol. A secure inference protocol enables users and model owners to interact so that the user obtains the prediction result while ensuring that neither party learns any other information about the user input or the model.

Liu et al. [27] designate Data Owners (DO) as the users who possess the DT training data, Cloud Service User (CSU) as a user with data to evaluate in the PPDT, and both Cloud Service Provider (CSP) and Evaluation Service Provider (ESP) as distinct cloud providers responsible for training and evaluating data, respectively. It is worth noting that both the CSP and the ESP must have knowledge of all the possible labels and attributes for the DT. They utilize the Distributed Two Trapdoors Public Key Cryptosystem (DT-PKC), which is similar to Paillier [33], with the distinction that it allows comparisons between ciphertexts encrypted by different public keys. Liu et al. [27] created new protocols to securely count the frequency of attributes in the data set, and modified Veugen's algorithm to be used for DT-PKC [45][46]. Our paper has the advantage of using 2048-bit keys, which are more secure than Liu et al. [27] and our PPDT has faster PPDT evaluation times.

Yuan et al. [52] utilize gradient-boosting decision trees (GBDT) to train DTs. They introduce the privacy-preserving distributed GBDT (PPD-GBDT), which uses differential privacy, polynomial approximation, and fully homomorphic encryption to achieve comprehensive privacy protection for their DT model. The DOs add noise to their local GBDT, and then convert the model into a polynomial format that is sent to the CSP. For evaluation, the client encrypts their input and sends the ciphertext and public key to the CSP. The CSP would return encrypted results, and the client would decrypt the classification. Our PPDT has generally faster PPDT evaluation times and no noise is added, which could potentially cause classification inaccuracies.

IV. Privacy-Preserving Drone Navigation Through Homomorphic Encryption for Collision Avoidance

A. Threat Model

We assume Alice and Bob, representing two drones, respectively, to be honest but curious, so we expect both parties to faithfully follow the encrypted intersection algorithm but will attempt to gather as much information as possible from each other such as path information, destination location, etc. We also assume that there may be external parties that will attempt to obtain the path information of the drones.

As an attack, we consider that Alice may attempt to replicate Bobs' path even if it is encrypted through intersections by defining her own path as a set of very small line segments that cover an entire area to a resolution that will give her Bob's path information, as well as a close replica of a complete path.

B. Proposed Approach

1) System Model and Scenario Overview: In our scenario, we assume that the two drones that are communicating with each other through our protocol are owned by different companies, for example Amazon and UPS. In this scenario, Alice is a drone about to take off, and Bob is either in flight or also ready for take-off. We consider a specific timeframe where Alice can send a request for paths of any other drones that are or will be flying in a neighborhood along with its speed. This would require communication between drones at longer distances than local area networks (e.g. WiFi-like ranges). This can be achieved by employing protocols such as 5G NB IoT [43] or LoRa [49]. This neighborhood should be much larger than the area covered during the flight to cover all drones that could come into the flight area during flight time. Bob, which is or will be flying in the area Alice is broadcasting to, responds. Note that we expect these drones to have different starting and delivery locations. Thus, it would be unlikely that the same pair of drones would possibly collide multiple times in an honest path. However, we assume that all drones have a default altitude enforced by regulations

2) Encrypted Comparison of Paths: We rely on homomorphic encryption to be able to compare the paths of two drones without exposing any information to each other regarding the path including any intersection point. In this approach, Bob has a key pair and a route composed of an arbitrary number of line segments. Alice also has a route composed of an arbitrary number of line segments. A sample route along with their line segments is shown in Fig. 1.

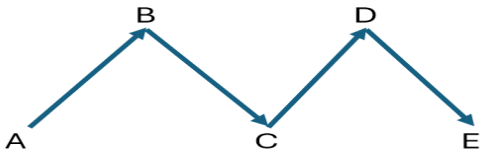

Fig. 1. A Path is compromised of an arbitrary number of lines segments, e.g. (A, B), (B, C), (C, D), (D, E)

Both Alice and Bob will follow the steps below to run our approach, as also shown in Fig. 2:

1) Bob will encrypt his route using his own public key and await a connection from Alice.
2) Upon connection to Alice, Bob sends his public key and encrypted route to Alice.
3) Alice then uses Bobs public key to encrypt her route, and they both complete an encrypted version of the route using Algorithm 2.

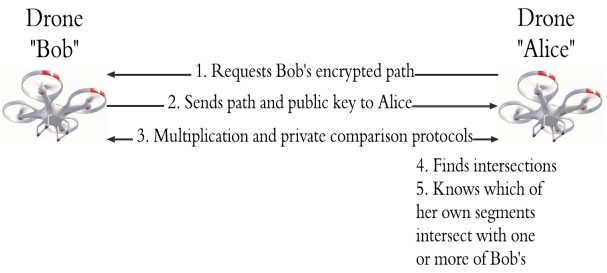

Fig. 2. Protocol for two drones to determine which intersections there might be collisions to avoid.

Once the algorithm ends, Alice knows which of her line segments on her path will collide with Bob. As Bob's path is encrypted, no other information is available to Alice. Bob does not receive the results of the comparisons and, if he wishes to obtain the same information, the protocol needs to be run again with the roles switched. However, this is not necessary, as it would be assumed that Alice would change altitude to avoid colliding with Bob.

Algorithm 1 Drone behavior in flight

Input: Protocol-Initiation-Range

```
while In Flight do
    if Another drone is in Protocol-Initiation-Range then
        Determine which Drone is Alice and Bob
        Segments ← Encrypted version of Algorithm 2
        for Segment in Path do
            if Collision occurs at Segment i then
                Alice adjusts altitude
            else
                Return to default altitude
            end if
        end for
    else
        return continue
    end if
end while
```

3) Computing Intersections: The line segment intersection algorithm [15] (that is, the Algorithm 2) that we use is based on the concept of orientation of an ordered triplet. There are three possible orientations for any ordered triplet of points: *clockwise*, *counterclockwise*, or *collinear*. To demonstrate this, we used the points (A, B, C), shown in their three possible orientations in Figure 3.

Assume that we have two line segments (A , B) and (C, D) that intersect, there are two cases that would occur.

In the general case, where two line segments intersect, (A, B, C) and (A, B, D) have different orientations and the orientations of (C, D, A) and (C, D, B) are also different. In the other case, if all line segments are collinear, they intersect if their x-projection and y-projections overlap. The pseudocode to find the orientation of an ordered triplet (A, B, C) made up of points (A_x, A_y), (B_x, B_y), and (C_x, C_y), where the subscript

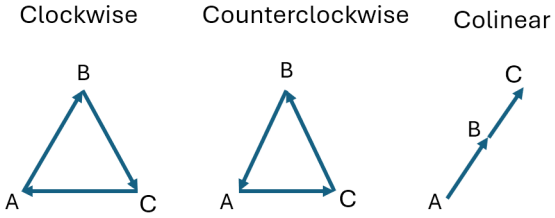

Fig. 3. Three cases of orientation for points A, B, and C.

identifies the plane of projection. To handle the case that two segments are collinear, we check if any of the four points is located in the other segment.

We can compose a routine that will decide whether two line segments (A, B) and (C, D) intersect. The pseudocode for this is provided in Algorithm 2.

Algorithm 2 Line Segment Intersection Algorithm [15]

```
O1 = Orientation(A,B,C)
O2 = Orientation(A,B,D)
O3 = Orientation(C,D,A)
O4 = Orientation(C,D,B)
if O1 ≠ 02 ∧ O3 ≠ 04 then
    return True
end if
if O1, O2, O3, O4 are all Collinear then
    if      A is on segment (C,D) ∨
    B is on segment (C,D) ∨
    C is on segment (A,B) ∨
    D is on segment (A,B) ∨ then
        return True
    else
        return False
    end if
end if
```

4) Security Considerations: Since the paths are encrypted, neither Alice nor Bob and no other third party can decrypt any information collected from the wireless broadcasts.

Now, let us assume Alice will be logging the results of the collision for each line segment in her path with Bob in an attempt to reverse engineer Bob's path. If there is no collision, privacy is preserved. If there is only one collision, Alice would know that one line segment would intersect with Bob. Since there is no time information, Alice would not be able to determine where in the line segment Bob would collide, or Bob's speed or direction.

During the engagement with Bob, should Alice decide to run the intersection protocol repeatedly against a series of closely spaced parallel lines composed of short segments, as shown in Figure 4, she could compose an approximation of Bob's path through the intersections she discovers.

The first mitigation of this attack is that, based on our assumptions in Section IV-A, the drones would have different starting points, so realistically it would still be difficult for Alice to randomly find Bob to attempt this sort of path. This

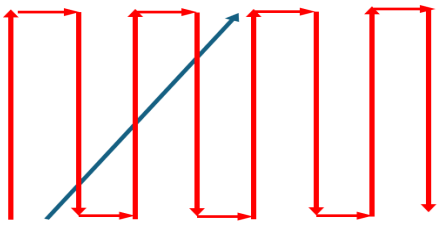

Fig. 4. The red path is the path of an Alice drone attempting to brute force, the blue path of a Bob drone.

attack also does not factor in that an Alice drone attempting this brute-force approach could just as easily miss fast-moving Bob drones [16] that came too early or too late within the long parallel line segments. This attack requires Alice and Bob to be within range of each other and to talk to each other for a very long time, which is not possible. Finally, drones also have a very limited range [4], so if Alice wanted to complete both her delivery and try this task, it would be risky, as the drone increases its risk of running out of energy before returning to its refueling station.

C. Performance Results

D. Experiment Setup

We implemented our protocol entirely in Java and used a homomorphic encryption library implemented in Java [37][36]. We open-sourced the code we used to implement our collision avoidance algorithm[1].

We considered the following metrics to access performance:

- *Execution Time*: This is the time for computing if an intersection would occur between two line segments
- *Network Traffic*: We count the number of bytes that both Alice and Bob write to a socket during the protocols.

Thirty random x-y plane line segments with values from -99 to 99 were generated because [22] does not provide intersection locations. To fairly compare speed, we used single segments, so both protocols returned the same result: a Boolean decision on the existence of an intersection. The experiment was completed on a pair of Debian 12 virtual machines with 2 CPUs and 4 GB of RAM, which were chosen to emulate the resources of a Raspberry Pi 4[2]. We calculated the average time and network traffic size for each comparison based on these 30 trials.

E. Experimental Results

1) Comparison with Li et al.: When comparing our work with Li et al.[22], we both used 2048-bit keys. The results in Table I indicate that our approach is 30% faster and sends less data over the network compared to Li et al. [22]. This is because of the efficiency of the comparison operation in DGK. In contrast, MPC-based approaches require a lot of communications and computation to perform comparison operations. The dominating nature of comparison operations in path comparison shows that any time savings should focus on this operation, and use of our approach supports this observation. The gains will even improve with an increase in the number of segments for a given path comparison since only a single segment is focused on average.

TABLE I
COMPARING THE COMPUTATION TIME AND BYTES SENT OVER A NETWORK WITH [22].

Approach	Time (seconds)	Network traffic (bytes)
Our Approach	4.407s	4634
Li et al. [22]	6.092s	39221

The improvements in speed and network traffic means that our approach would be scalable for multiple pairs of drones to compare their paths within a certain airspace. Our protocol would have the edge if used in networks where bandwidth is limited, such as when using LoRa.

2) Comparison with Desai et al.: We also performed an experiment to compare our approach with Desai et al. We used two Raspberry Pi 4s to have a fair comparison (as done in that work). We obtained an average run-time of 16.531 seconds, which is faster than Dessai et al. [13] taking 30 seconds for a matrix of size 96 and using a secret share size of 64 bits.

It is important to understand that the speed of our approach and that of Dessai et al. [13] vary based on different parameters. Route complexity does not change the speed of Dessai et al.'s [13] approach as it does ours, but the area and resolution considered for route comparisons does. But a matrix size of 96 is very limiting when we consider realistic drone flight areas, and Desai et al. [13] computation time scales exponentially with increased matrix size.

F. Conclusion and Future Work

In this paper, we implement a new encrypted intersection algorithm using homomorphic encryption, which can be used by two drones owned by separate entities to compute where they would intersect. Our algorithm is more flexible than [13] as we can use GPS coordinates as part of our line segments, meaning more flexibility on where the protocol could work, as well as not requiring all participating drones to share a finite possible flight area. Furthermore, the results of the experiment showed that our algorithm outperforms the existing approaches. It is faster than both Li et al.'s [22] collision protocol and Desai et al. MPC protocol and requires less network bandwidth.

The primary privacy concern, a brute force attack designed to reconstruct a flight path out of detected collision points, would take far longer than the flying drone would be in the air, and would fail due to lost connectivity between the drones. The large difference in computation time between an honest path and a brute-force attack also leaves open the possibility of stopping the attack based on assumptions about time and resources used in an honest implementation.

The aforementioned use of GPS coordinates as input allows for the potential for our approach to be used in-flight in real-time, as opposed to route planning. Using this approach in this

[1]https://github.com/adwise-fiu/homomorphic-path-comparison
[2]https://www.raspberrypi.com/products/raspberry-pi-4-model-b/

way is currently limited by computation time, but as drones increase in computation power and wireless networks increase in bandwidth, it is possible that this approach be used in an in-flight scenario.

V. Enhanced Outsourced and Secure Inference for Tall Sparse Decision Trees

A. System Model

We assume the following entities. A **Client** has a feature vector $\boldsymbol{x}$ and wants to classify it using the DT. The client creates public and private Paillier [33] and DGK [10][11][12] keys. The client will give the server the public keys. *In a real-world scenario, a client could be a doctor who needs to know if a patient has thyroid disease*. A **Server** has access to the training data and creates the DT. The server creates the PPDT to send to the level-sites. To save network bandwidth, the server will give the level-sites the public keys. *In a real-world scenario, this could be a laboratory that can inform a doctor if there is a probable sign of thyroid disease.* A **Level-site** only has a level of the PPDT and the public keys used to compare integers with the client. *In a real-world scenario, a level-site would be a cloud provider hosting the PPDT.*

B. Threat Model

We assume the server is trusted, but the client and level-sites are "honest-but-curious" (HBC). The client may attempt to learn the decision nodes of a DT, which would contain the feature and a corresponding threshold. We assume that the level-sites will attempt to learn the client's feature vector data and classification output.

We expect the level-sites to strictly follow the protocol of our designed PPDT. However, we will assume that the level-sites would be curious to analyze both power consumption and computation times for potential side-channel attacks [3][2]. We also consider a passive adversary, $\mathcal{A}$, outside the system in our model. $\mathcal{A}$'s goal is to learn the input of the client, the classification of the PPDT model, and the thresholds of the PPDT model.

C. PPDT Approach

1) Goals and Overview: We are motivated by the fact that in a tall and sparse decision tree, most of the leaves in a DT are not at the maximum depth of the tree. Indeed, we computed the level of each classification from the training dataset [3] as shown in Table II. We want to minimize the encrypted comparisons used to improve our performance for our PPDT in this circumstance. In addition, if a level-site l fails, we would like the traffic redirected to a backup, solving the single point of failure in Joye and Salehi's approach [20].

Based on these goals, we set up our PPDT as follows: **(i)** The server builds the DT with the training data; **(ii)** The client sends to the server the DGK [10][11][12] and Paillier [33] public key. The server provides the client with both the classes of the DT; and **(iii)** For each level 0, 1, $\cdots$, $d-1$, where d is the depth of the DT, the server sends the **encrypted** thresholds and classifications to the respective level-site.

[3]https://github.com/renatopp/arff-datasets

TABLE II
Analysis of level of classifications of Training data

Dataset	Average	Median	3rd Quartile	Max	Size
Hypothyroid	2.78	2	2	9	3372
Spambase	9.60	9	11	22	4601
Nursery	5.99	7	8	14	12960

2) PPDT Inference: In our setting, there is one client and d level-sites, where d is equivalent to the depth of the DT. The client has a private feature vector, $\boldsymbol{x} = (x_1, x_2, \cdots, x_n) \in \mathbb{Z}^n$, which is encrypted using homomorphic encryption to generate $[\![\boldsymbol{x}_i]\!]$, where $i \in 1, \cdots, n$. We also assume that all thresholds are t-bits long. Each level site uses the Joye and Salehi encrypted integer comparison protocol [20] or a timing attack resistant encrypted equality protocol [31] for categorical data. We note that in our PPDT, *we improve our PPDT performance by not using oblivious transfer or converting to a complete binary tree as in the original work* [20], as shown in Figure 5.

3) Level-site Evaluation: We evaluated the PPDT by traversing the level by level, where the level-site l sends to the level-site $l+1$ the encrypted index i of the previous level-site. The client encrypts $\boldsymbol{x}$ and sends it to level-site 0.

Consider l to be the level of the tree we are currently considering and k to be the index of the current node at that level. We say M_k^l is the blinded threshold comparison value at level l at position k and $[\![M_k^l]\!]$ to be the encrypted blinded threshold comparison value.

We have to compute at each level-site l the quantity $[\![M_k^l]\!] = [\![x_i - T_k^l + \mu_l]\!]$. All these calculations are performed without decryption by using additively homomorphic encryption since: $[\![M_k^l]\!] = [\![x_i - T_k^l + \mu_l]\!] = [\![x_i]\!] \boxplus [\![-T_k^l]\!] \boxplus [\![\mu_l]\!]$. We note that μ_l is a random $(t+\kappa)$ bit mask computed at the level-site l, where κ is a security parameter [20]. In addition, $[\![x_i]\!]$ is the value of an attribute of $\boldsymbol{x}$ matched with the attribute of T_k^l.

Both the client and the level site l utilize [31] to compare categorical data or [20] to compare numerical data. We used a label encoder so that we can use an encrypted equality check to traverse a decision tree. The encrypted comparison protocols require M_k^l and μ_l as inputs. At the end of the protocol, the level-site l has δ_l and the client has $\delta_l^{'}$, such that:

$$\delta_l \oplus \delta_l^{'} = \begin{cases} Numerical & \mathbb{1}\{T_k^l \leq x_i\} \\ Categorical & \mathbb{1}\{T_k^l == x_i\} \end{cases}$$

Without the other corresponding bit, neither the level-site l nor the client knows the value of the inequality. To avoid this, the level-site l can additively blind δ_l with $r^{'}$ and send it to the client to compute $(\delta_l + r^{'}) \oplus \delta_l^{'}$. This output is returned to level-site l, as level-site l knows $r^{'}$, it can determine the inequality output without revealing δ_l to the client. The output of the inequality informs how to update the index k for the level-site $l+1$.

If the node at index k at the level-site l is a leaf, level-site l returns the encrypted classification to the client. We summarize our approach in Algorithm 3.

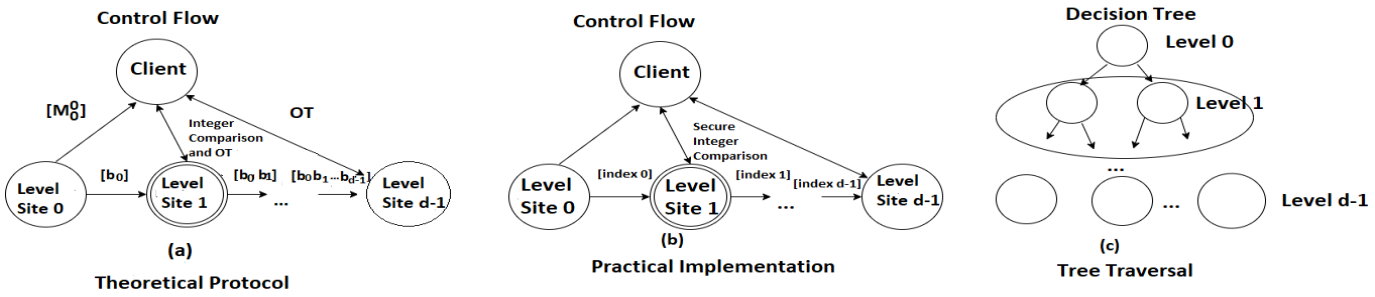

Fig. 5. (a) Direct conversion of the Joye and Salehi PPDT [20] to the level-site approach. (b) Control flow in our PPDT implementation, and (c) Tree Traversal. We emphasize Level 1 of the DT evaluation

.

Algorithm 3 PPDT Secure Inference Protocol with level-sites

```
next-index=0
Client sends x and next-index to level-site 0
for int l = 0; l < d; l++ do
    level-site l receives indexes and x
    for int k = 0; k < l.NodeCount(); k++ do
        if Not in-scope based on next-index then
            continue
        else if in-scope and node i is a leaf then
            return encrypted-classification to client
        else
            secure comparison w/ client and level-site l
            next-index ← set index for next level-site
            level-site l sends index and x to level-site l+1
            break
        end if
    end for
end for
```

D. Security Analysis

We construct security proofs via simulation, reducing our protocol to multi-party integer comparisons and homomorphic encryption, and citing original proofs. Assuming an honest-but-curious (HBC) adversary, we find that it can gather information on the PPDT through numerous queries. We then discuss countermeasures. We conclude by demonstrating how our PPDT is safeguarded against side-channel attacks and can be upgraded for post-quantum security.

Our proofs use simulation. We consider a PPDT classification protocol, $f(x,y)$, which we break into a two-party protocol such that $f(x,y) = f_1(x,y), f_2(x,y)$, and consider our implementation, π, of the functionality. During the execution of π, each of the two parties will generate a $view_i^{\pi}, \forall i \in \{1,2\}$. We claim that our system is secure if we can show that there exist two simulators, $\mathcal{S}_1(1^n, x, f_1(x,y))$ and $\mathcal{S}_2(1^n, y, f_2(x.y))$ that are computationally equivalent to $view_1^{\pi}(x,y,n)$ and $view_2^{\pi}(x,y,n)$ where n is a security parameter.

1) Security against an HBC client: We assume a HBC client who follows the protocol faithfully but wishes to learn information about the tree node. To demonstrate that this client does not learn anything more than its view, we generate a simulator $\mathcal{S}_1$ that takes the input of the client and simulates the output of a level-site.

If the level-site l is equal to zero, the simulator must sample a value $M_0^0 \xleftarrow{R} \mathbb{Z}_p^*$, and encrypt it to achieve $[\![M_0^0]\!]$. This number is computationally indistinguishable from any real input that could be sent by a level-site given the properties of additive homomorphic encryption as demonstrated in [39]. The client will then decrypt this number, resulting in M_0^0, and define $M_0' = M_0^0$ and $m_0' = M_0^0 \bmod 2^t$. To simulate the integer comparison protocol, it will then sample $b_r \xleftarrow{R} \{0,1\}$. This value will take the place of b_0[4]. As a result of the integer comparison protocol, each output in this case is equally likely to meet the standard of computational equivalence.

In the case that the level-site is greater than 0, we need to perform more steps. First, the simulator sets $j \leftarrow (b_0', \cdots, b_{l-1}')_2$, according to the protocol. As before, the simulator samples a value $M_k^l \xleftarrow{R} \mathbb{Z}_p^*$ and follows the protocol by encrypting a randomly sampled value to receive $[\![M_k^l]\!]$, simulating the response from the level-site. The next level-site is then passed the encrypted client feature vector and the node index with which to compare. This process repeats until the simulator comes to a classification.

This simulator simply performs the same steps as the client, and randomly draws values to simulate the secret parameters of the level-site. Given that the comparison protocol and the additive homomorphic encryption scheme are statistically indistinguishable from the random output [45], [46], [31], [39], nothing about the internal state of the level-site is revealed through these operations.

2) Security against an HBC level-site l: We assume the existence of HBC level-sites that wish to learn the client's private input vector x. We construct a simulator that produces computationally indistinguishable output from a legitimate run of the system.

The simulation begins when the simulator sets $x_i = 0, \forall x_i \in x$, generates a key pair (pk, sk) and encrypts the x_i's. The simulator then runs the protocol from the level-site by generating μ_l, σ_l, k, s, etc., and generates $[\![M_k^l]\!]$. Then it generates j

[4]In our practical implementation, we use integers or indexes instead of bit strings that denote a path down the tree. However, this is functionally equivalent.

by randomly selecting bits $(b'_0, \cdots, b'_{l-1})_2$. The simulator then goes through the comparison portion of the protocol. The level-site l passes on $[\![b_0, \cdots, b_l]\!]$ and the encrypted feature vector to level-site $l+1$.

Simply, the simulator runs the level-site's protocol and encrypts the value 0 for $x_i, \forall x_i \in$ feature vector $\boldsymbol{x}$. It then steps through the protocol and, due to the encrypted integer comparison protocol, does not learn any information about the value of x_i. This makes it computationally indistinguishable from a real run of the system. Level-site l uses the previous results of the tree-traversal for the previous level-sites, $0, \cdots, l-1$ the required x_i's for the current level l the thresholds for the current level l. Only the client knows the feature vector $\boldsymbol{x}$ and its classification. Thus, the level-site does not learn any relevant information from the client.

3) Client Timing Attack and Performance Tradeoffs: Over numerous runs, the client can begin to estimate the thresholds stored in different nodes by observing different feature vectors and classification results. An easy remediation is to throttle the client, as is done by other PPDT models [27]. However, the client can determine the level on which the class is on using the amount of time that a classification takes. This attack can be mitigated by utilizing a proxy between the client and level-sites that adds random waiting times to the query result. The minimum extra time needed would be between 0.5 and 1 seconds to make it indistinguishable between two adjacent level-sites, as shown in Table III. Throttling would not severely affect the average run-time compared to the original Joye and Salehi PPDT protocol [20].

4) Security against Side Channel Attacks: Side channel attacks are possible on multiple cryptosystems such as RSA [3]. We use Joye and Salehi's [20] comparison protocol for comparing numerical attributes, which is timing attack resistant on each level-site. For categorical data comparison, we used a modified *Protocol 1 EQT-1* [31] that always runs the same computations, which also makes it resistant to timing attacks. As both comparison protocols also have a similar run-time, the overall system has timing attack protection. Thus, the computation time between all the level-sites will be indistinguishable. If a classification finishes early, we can send a bogus value downstream to ensure that all level-sites complete a computation.

We use containerization to ensure stronger mitigations against power-based side-channel attacks. Containers are immutable after being created [1], making it difficult to install software to analyze the runtime environment. Since containers are usually the end result of an automated deployment pipeline, consistent replacement of keys at level-sites [21] would also be a mitigation. More, a data owner could split the level-sites to be managed by multiple non-colluding cloud providers, which would make obtaining power readings to derive the private key much more difficult.

E. Experimental Results

1) Experiment Setup: We implemented and tested the proposed approach using Amazon Elastic Kubernetes Service (EKS). We used t2.medium EC2 instances, which have 2 vCPUs and 4 GB of RAM. Our open source code implementation[5] used the homomorphic encryption library implemented in Java [37] that implements Joye and Salehi's encrypted integer protocol [20] and Veugen's encrypted equality comparison [31].

We used Docker containers and Kubernetes as it allows the creation of replicas [17] and supports horizontal scaling [35], which fits our availability requirements in Section V-C1. We considered two EC2 VMs in the same region but in a different availability zone to simulate using two non-colluding cloud providers.

2) Performance Results: For our experiments, we ran our PPDT with each container storing a level-site. When obtaining our results, we did not implement throttling based on the discussion in Section V-D3, as we wanted to have a consistent view on how much time was needed for the PPDT evaluation at each level-site. We ran the evaluation 10 times each and provided the average execution time. The client experienced on average a 25-millisecond transmission delay for each connection, which is considered in the results. We also report the estimated time that Joye and Salehi PPDT would take by computing an evaluation that would traverse the d levels of the PPDT.

Our solution agrees 100% with a standard DT classification. Setting up the level-sites was completed in less than a second for all datasets. The performance results of our approach compared to Joye and Salehi are reported in Table III. Our superior performance is due to not requiring a complete binary tree [20], reducing the evaluation depth and the number of encrypted comparisons.

TABLE III
COMPARING CLASSIFICATION TIME OF OUR APPROACH WITH JOYE AND SALEHI APPROACH UNDER HYPOTHYROID AND NURSERY DATASETS.

Levels	Hypothyroid level-sites	Hypothyroid Joye & Salehi	Nursery level-sites	Nursery Joye & Salehi
2	**0.807 s**	3.279 s	**0.656 s**	4.140 s
4	**1.772 s**	3.279 s	**2.041 s**	4.140 s
9	4.148 s	3.279 s	**2.950 s**	4.140 s
12	N/A	N/A	5.648 s	4.140 s

Based on Table II, the average depth for the Hypothyroid set is 2.78 which corresponds to a value between 0.807 and 1.772 seconds in Table III. This is significantly faster than the Joye and Salehi approach. Similarly, the average depth for Nursery data is 5.99 which corresponds to a value between 2.041 and 2.950 seconds, again much less than 4.140 of Joye and Salehi.

3) Comparison with other Related Work: We compared our approach with Li et al. [27] and when repeating the same client queries, it takes our PPDT 2.041, 6.198 and 15.878 seconds for the nursery, breast cancer, and spambase dataset, respectively, which is at least 33% faster. Compared to Yuan et al. [52] using the *spambase* dataset, which they reported for evaluations. As shown in Table IV, our method is faster at almost all levels due to Yuan et al.'s need to encrypt data for each evaluation and the computational overhead of traversing the PPD-GBDT.

[5]https://github.com/adwise-fiu/Level-Site-PPDT

TABLE IV
COMPARING THE CLASSIFICATION TIME OF [52] WITH OUR APPROACH USING SPAMBASE DATASET.

Levels	Our Approach	Yuan et al.[52]
2	1.927 s	0.49 s
3	2.736 s	7.58 s
4	3.937 s	8.35 s
5	4.638 s	17.33 s
6	5.783 s	19.37 s

F. Conclusion

In this paper, we present a new secure decision tree inference protocol by splitting the DT model into level-site constructs. Our PPDT exhibits optimal performance in sparse trees [19][24] due to fewer encrypted integer comparisons, resulting in faster performance compared to other PPDTs [27][52][20]. In addition, it offers resistance to timing attacks through timing attack resistant comparison protocols [20][31]. Our PPDT also offers strong mitigations against power-based side channel attacks [3], as we use containers that offer power-based side channel protections [1], key replacement [21], and our approach can be used to split our PPDT with two non-colluding cloud providers. Finally, our PPDT protocols allow us to support load balancing to more evenly distribute more computing resources to higher levels of the PPDT, which would experience more usage and redundancy [17] in case a level-site experiences an outage.

VI. CONCLUSION

In this work, we have addressed critical challenges in secure and privacy-preserving computation, demonstrating the transformative potential and practical versatility of homomorphic encryption. Through two distinct but complementary research efforts, we have showcased how a single powerful HE library can be effectively applied to enable secure operations in diverse real-world scenarios.

Specifically, in our work on privacy-preserving drone navigation [28], we developed a novel collision avoidance algorithm that uses HE-based comparisons. This approach not only eliminated the need for computationally expensive pre-processing methods (unlike [13]) but also exhibited significant performance advantages in computation time and network bandwidth compared to prior art [22]. Furthermore, our research on outsourced and secure enhanced inference for large, sparse decision trees [38] introduced a new PPDT evaluation protocol. This protocol minimizes encrypted integer comparisons, leading to improved performance over existing methods [52][27], while simultaneously bolstering resilience against single points of failure by enabling distributed model sharing across non-colluding cloud providers.

Crucially, the successful implementation of both of these complex applications utilizing the **same underlying homomorphic encryption library** [37] serves as a testament to the technology's adaptability and maturity. This work underscores that modern HE libraries can be seamlessly integrated into diverse applications, providing robust privacy controls without necessitating fundamental rearchitectures. This versatility positions homomorphic encryption as a foundational technology for building the next generation of secure and privacy-aware systems, from autonomous vehicles to cloud-based machine learning.

REFERENCES

[1] NIST Special Publication 800-190. Application Container Security Guide. https://csrc.nist.gov/pubs/sp/800/190/final, September 2017. [Online; accessed 29-December-2023].

[2] Furkan Aydin, Emre Karabulut, Seetal Potluri, Erdem Alkim, and Aydin Aysu. Reveal: Single-trace side-channel leakage of the seal homomorphic encryption library. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1527–1532. IEEE, 2022.

[3] Alessandro Barenghi, Diego Carrera, Silvia Mella, Andrea Pace, Gerardo Pelosi, and Ruggero Susella. Profiled side channel attacks against the rsa cryptosystem using neural networks. *Journal of Information Security and Applications*, 66:103122, 2022.

[4] Sherilyn Beall. What is the range on a drone. https://robots.net/tech/what-is-the-range-on-a-drone/. [Online; accessed 23-May-2024, Published; 19-October-2023, Modified; 28-February-2024].

[5] Fabio Borghetti, Claudia Caballini, Angela Carboni, Gaia Grossato, Roberto Maja, and Benedetto Barabino. The use of drones for last-mile delivery: A numerical case study in milan, italy. *Sustainability*, 14(3), 2022.

[6] Raphael Bost, Raluca Popa, Stephen Tu, and Shafi Goldwasser. Machine learning classification over encrypted data. In *Network and Distributed System Security Symposium (NDSS 2015)*. The Internet Society, 01 2015.

[7] Tiziana Calamoneri, Federico Corò, and Simona Mancini. A realistic model to support rescue operations after an earthquake via uavs. *IEEE Access*, 10:6109–6125, 2022.

[8] Gabriel Carrasco-Escobar, Marta Moreno, Kimberly Fornace, Manuela Herrera-Varela, Edgar Manrique, and Jan E. Conn. The use of drones for mosquito surveillance and control. *Parasites and Vectors*, 15(1):473, 2022.

[9] Christa Cody, Vitaly Ford, and Ambareen Siraj. Decision tree learning for fraud detection in consumer energy consumption. In *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, pages 1175–1179, 2015.

[10] Ivan Damgrd, Martin Geisler, and Mikkel Krigaard. Efficient and secure comparison for on-line auctions. In *Australasian conference on information security and privacy*, pages 416–430. Springer, 2007.

[11] Ivan Damgrd, Martin Geisler, and Mikkel Krigaard. Homomorphic encryption and secure comparison. *International Journal of Applied Cryptography*, 1(1):22–31, 2008.

[12] Ivan Damgård, Martin Geisler, and Mikkel Kroigard. A correction to 'efficient and secure comparison for on-line auctions'. *International Journal of Applied Cryptography*, 1(4):323–324, 2009.

[13] Anushka Desai, Oscar G. Bautista, and Kemal Akkaya. Privacy-preserving collision detection for drone-based aerial package delivery using secure multi-party computation. In *Proceedings of the Twenty-Fourth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, MobiHoc '23, page 510–515, New York, NY, USA, 2023. Association for Computing Machinery.

[14] Fatih Emekçi, Ozgur Sahin, Divyakant Agrawal, and Amr Abbadi. Privacy preserving decision tree learning over multiple parties. *Data and Knowledge Engineering*, 63:348–361, 11 2007.

[15] Cormen Thomas H. *Introduction to Algorithms*. PHI Learning Pvt. Ltd., 2009.

[16] La Verne Haun. How fast can a delivery drone fly. https://robots.net/tech/how-fast-can-a-delivery-drone-fly/. [Online; accessed 23-May-2024, Published; 20-October-2023, Modified; 02-March-2024].

[17] Michael Hausenblas. Kubernetes Replicas: Underappreciated Workhorses. https://www.redhat.com/en/blog/kubernetes-replicas-appreciated-workhorses, July 25, 2017. [Online; accessed 29-December-2023].

[18] Health and Human Services. Health insurance portability and accountability act. https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html, 1996.

[19] Xiyang Hu, Cynthia Rudin, and Margo Seltzer. Optimal sparse decision trees. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32, pages 7265–7273. Curran Associates, Inc., 2019.

[20] Marc Joye and Fariborz Salehi. Private yet efficient decision tree evaluation. In *Proceedings of DBSec 2018*, pages 243–259, 07 2018.

[21] Paul Kocher, Joshua Jaffe, Benjamin Jun, and Pankaj Rohatgi. Introduction to differential power analysis. *Journal of Cryptographic Engineering*, 1:5–27, 2011.

[22] Li Li, Alfredo Bayuelo, Leonardo Bobadilla, Tauhidul Alam, and Dylan A. Shell. Coordinated multi-robot planning while preserving individual privacy. In *International Conference on Robotics and Automation (ICRA)*, pages 1–7, 2018.

[23] Hsiao-Ying Lin and Wen-Guey Tzeng. An efficient solution to the millionaires' problem based on homomorphic encryption. In *International Conference on Applied Cryptography and Network Security, ACNS 2005*, pages 456–466, 06 2005.

[24] Jimmy Lin, Chudi Zhong, Diane Hu, Cynthia Rudin, and Margo Seltzer. Generalized and scalable optimal sparse decision trees. In *International Conference on Machine Learning*, pages 6150–6160. PMLR, 2020.

[25] Yehuda Lindell and Benny Pinkas. Privacy preserving data mining. In Mihir Bellare, editor, *Advances in Cryptology - CRYPTO 2000*, pages 36–54, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.

[26] Yehuda Lindell and Benny Pinkas. Secure multiparty computation for privacy-preserving data mining. *Journal of Privacy and Confidentiality*, 1:197, 11 2008.

[27] Lin Liu, Rongmao Chen, Ximeng Liu, Jinshu Su, and Linbo Qiao. Towards practical privacy-preserving decision tree training and evaluation in the cloud. *IEEE Transactions on Information Forensics and Security*, 15:2914–2929, 2020.

[28] Allan Luedeman, Nicholas Baum, Andrew Quijano, and Kemal Akkaya. Privacy-preserving drone navigation through homomorphic encryption for collision avoidance. In *2024 IEEE 49th Conference on Local Computer Networks (LCN)*, pages 1–6. IEEE, 2024.

[29] Ignacio Martinez-Alpiste, Gelayol Golcarenarenji, Qi Wang, and Jose Maria Alcaraz-Calero. Search and rescue operation using uavs: A case study. *Expert Systems with Applications*, 178:114937, 2021.

[30] Hefeng Mo, Ting Tao, Wenbin Gao, Chunlei Fan, Ligang Wang, Xin Zhao, and Haili Wang. Application of decision tree algorithm in medical nursing and health assessment system. In *2023 3rd Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS)*, pages 55–59, 2023.

[31] Majid Nateghizad, Thijs Veugen, Zekeriya Erkin, and Reginald L Lagendijk. Secure equality testing protocols in the two-party setting. In *Proceedings of the 13th International Conference on Availability, Reliability and Security*, pages 1–10, 2018.

[32] National Science Foundation News. The future of encryption. https://www.youtube.com/watch?v=BylWT5gsgfM. [Online; accessed 15-Jun-2025].

[33] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Advances in Cryptology—EUROCRYPT'99: International Conference on the Theory and Application of Cryptographic Techniques Prague, Czech Republic, May 2–6, 1999 Proceedings 18*, pages 223–238. Springer, 1999.

[34] European Parliament and Council of the European Union. General data protection regulation. https://gdpr.eu/, 2019.

[35] Bjørn Erik Pedersen. Horizontal Pod Autoscaling. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, May 5, 2018. [Online; accessed 29-December-2023].

[36] Andrew Quijano. Homomorphic encryption library. https://github.com/adwise-fiu/Homomorphic_Encryption, 2019.

[37] Andrew Quijano and Kemal Akkaya. Server-side fingerprint-based indoor localization using encrypted sorting. In *2019 IEEE 16th International Conference on Mobile Ad Hoc and Sensor Systems Workshops (MASSW)*, pages 53–57, Monterrey, CA, 2019. IEEE.

[38] Andrew Quijano, Spyros T Halkidis, Kevin Gallagher, Kemal Akkaya, and Nikolaos Samaras. Enhanced outsourced and secure inference for tall sparse decision trees. In *2024 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, pages 1–6. IEEE, 2024.

[39] Rivest RL, Adleman LM, and Dertouzos ML. On data banks and privacy homomorphisms. *Foundations of Secure Computation*, 4:169–179, 01 1978.

[40] Savio Sciancalepore and Dominik Roy George. Privacy-preserving trajectory matching on autonomous unmanned aerial vehicles. In *Proceedings of the 38th Annual Computer Security Applications Conference*, ACSAC '22, page 1–12, New York, NY, USA, 2022. Association for Computing Machinery.

[41] Raymond Tai, Jack Ma, Yongjun Zhao, and Sherman Chow. Privacy-preserving decision trees evaluation via linear functions. In *Computer Security - ESORICS 2017, Part II, LNCS vol. 10493*, pages 494–512. Springer, 09 2017.

[42] Stacey Truex, Ling Liu, Mehmet Gursoy, and Lei Yu. Privacy-preserving inductive learning with decision trees. In *6th International Congress on Big Data*, pages 57–64, 06 2017.

[43] Hanif Ullah, Nithya Gopalakrishnan Nair, Adrian Moore, Chris Nugent, Paul Muschamp, and Maria Cuevas. 5g communication: An overview of vehicle-to-everything, drones, and healthcare use-cases. *IEEE Access*, 7:37251–37268, 2019.

[44] Thijs Veugen. Encrypted integer division. In *2010 IEEE International Workshop on Information Forensics and Security*, pages 1–6. IEEE, 2010.

[45] Thijs Veugen. Improving the dgk comparison protocol. In *WIFS 2012 - Proceedings of the 2012 IEEE International Workshop on Information Forensics and Security*, pages 49–54, 12 2012.

[46] Thijs Veugen. Correction to "improving the dgk comparison protocol". *Cryptology ePrint Archive*, 2018.

[47] Baptiste Vinh Mau and Koji Nuida. Correction of a secure comparison protocol for encrypted integers in ieee wifs 2012 (short paper). In *Advances in Information and Computer Security: 12th International Workshop on Security, IWSEC 2017, Hiroshima, Japan, August 30–September 1, 2017, Proceedings 12*, pages 181–191. Springer, 2017.

[48] Sophia Yakoubov. A gentle introduction to yao's garbled circuits. *preprint on webpage*, 2017.

[49] Evşen Yanmaz, Saeed Yahyanejad, Bernhard Rinner, Hermann Hellwagner, and Christian Bettstetter. Drone networks: Communications, coordination, and sensing. *Ad Hoc Networks*, 68:1–15, 2018.

[50] Wang Yanxia. Student information management decision system based on decision tree classification algorithm. In *2022 IEEE 5th International Conference on Information Systems and Computer Aided Education (ICISCAE)*, pages 827–831, 2022.

[51] Andrew C. Yao. Protocols for secure computations. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 160–164. IEEE, 1982.

[52] Shuai Yuan, Hongwei Li, Xinyuan Qian, Meng Hao, Yixiao Zhai, and Xiaolong Cui. Toward efficient and end-to-end privacy-preserving distributed gradient boosting decision trees. In *ICC 2023-IEEE International Conference on Communications*, pages 1566–1571. IEEE, 2023.